



Create Kubernetes Manifests



Nabil Ahmed

```
--
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
~
```



Introducing Today's Project!

In this project, I will set up manifest files because they are a key part of the Kubernetes deployment process. Manifest files give Kubernetes the instructions on how it should deploy your containerised application.

Tools and concepts

I used Amazon EKS, eksctl, Git, Docker, ECR to create a Kubernetes cluster and build a container image of an app to deploy. Key concepts include using manifests, Deployments and Services to get ready for Kubernetes deployment.

Project reflection

I chose to do this project today because learning about Kubernetes with AWS is a key skill for DevOps engineers. Something that would make learning with NextWork even better is nothing.

This project took me approximately 1 hour and 30 minutes. The most challenging part was understanding/breaking down the Deployment manifest. My favourite part was being able to set up manifest files and understand how they work together.



Project Set Up

Kubernetes cluster

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included granting my EC2 instance permissions over IAM, and installing eksctl command line tool. I created the cluster using eksctl create command.

Backend code

I retrieved the backend that I plan to deploy by pulling the code from a GitHub repository. Backend is the logic of an application. For example, how an app stores, retrieves and processes data.

Container image

Once I cloned the backend code, I built a container image because Kubernetes requires apps to be in container images to deploy them. I used Docker to build a container image of the backend code.

I also pushed the container image to a container registry, because this helps to establish a single source of truth for the latest version of my container image. To push the image to ECR, I ran the push commands that built and tagged the image.



Manifest files

Kubernetes manifests are instructions for Kubernetes on how it should deploy an app. Manifests are helpful because they reduce manual work when you are wanting to deploy and manage applications across multiple containers (clusters).

A Kubernetes deployment manages multiple copies of the same containerised backend. The container image URL in my Deployment manifest tells Kubernetes the attributes of the container that I'd like Kubernetes to deploy.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 840032438341.dkr.ecr.eu-west-2.amazonaws.com/nextwork-flask-backend:latest
          ports:
            - containerPort: 8080
```



Service Manifest

A Kubernetes Service exposes your application to network traffic. You need a Service manifest because the Deployment manifest takes care of creating/managing the application except how users/other apps will get access to them. Having no Service manifest means that there will be no traffic.

My Service manifest sets up a NodePort and a targetPort. It maps traffic that get to port 8080 of the Service to port 8080 of the container running the application.

```
--
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
~
```



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

